ParkingHub

Sistema de reserva de vagas de estacionamento
Guia para a apresentação do projeto

Equipe

Paulo Henrique Alves de Almeida

Matheus Avanzo dos Santos

Murilo Roselini

Petterson Augusto Papa de Souza

Raphael Souza Araujo Barbosa

Tecnologias: HTML, CSS e JavaScript puros, sem servidor e sem bibliotecas.

1. O que é o projeto

O ParkingHub é um site onde a pessoa **reserva uma vaga de estacionamento**, como se escolhesse o assento em um cinema. Ela escolhe o período, clica numa vaga da planta, informa seus dados e escolhe a forma de pagamento. Depois pode acompanhar, cancelar ou encerrar a reserva.

É um **protótipo**: não existe servidor nem banco de dados, e nenhuma cobrança real é feita. Tudo é salvo no próprio navegador (`localStorage`), então os dados sobrevivem ao recarregar a página, mas ficam só naquele computador.

Como abrir

Dê um duplo clique em `index.html`. Não precisa instalar nada nem estar online.

Arquivos do projeto

Arquivo	Para que serve	Linhas
`index.html`	**Estrutura**: o que existe na página (formulários, planta, lista, modal).	185
`style.css`	**Aparência**: cores, temas claro/escuro, desenho da planta, layout.	171
`script.js`	**Comportamento**: regras, cálculos, validações, salvamento.	493

> **Ideia central para explicar:** o HTML diz *o quê*, o CSS diz *como parece* e o JavaScript diz *o que acontece*. Os três trabalham juntos, ligados pelos `id` dos elementos.

2. O que o sistema faz

Recurso	Como funciona
Modalidades	**Por horário**: R$ 8,00 por hora (hora iniciada conta como hora inteira). **Diária**: R$ 35,00 por 24 horas.
Planta com 34 vagas	26 de carro (V1 a V26), 6 de moto (M1 a M6) e 2 acessíveis (D1 e D2).
Disponibilidade	Uma vaga fica bloqueada se já existe reserva ativa nela em um horário que se sobrepõe ao escolhido.
Regra da placa	A mesma placa não pode ter duas reservas ativas ao mesmo tempo, nem em vagas diferentes.
Vaga acessível	Exige o anexo de uma credencial (JPG, PNG, WebP ou PDF, até 5 MB). O arquivo é só conferido, não é enviado nem guardado.
Pagamento	Cartão e dinheiro ficam **pendentes** (paga no estacionamento). Google Wallet e Pix são **simulados**. Nunca há cobrança real.
Cancelar	A reserva vira "cancelada" e a vaga volta a ficar livre.
Simular saída	Informa quando a pessoa saiu. Cada hora de atraso (iniciada) custa R$ 5,00 a mais. A reserva vira "encerrada" com o total final.
Tema	Claro ou escuro, pelo botão no cabeçalho. Começa de acordo com o sistema do computador.

3. O fluxo do usuário

Etapa 1	**Etapa 2**	**Etapa 3**
Vaga e período (escolhe data, horário e clica na vaga)	Seus dados (nome, telefone, placa, credencial)	Pagamento (escolhe a forma e confirma)

A reserva **só é criada no último passo**. Se a pessoa voltar antes de confirmar, nada é salvo. As etapas 2 e 3 compartilham a mesma tela: muda apenas qual formulário aparece ao lado do resumo.

Ao confirmar, a pessoa volta para a etapa 1, vê a mensagem de sucesso e a nova reserva aparece em "Minhas reservas".

4. O arquivo index.html

Contém todo o conteúdo da página, organizado em blocos. O JavaScript encontra cada elemento pelo seu `id`.

Bloco	O que é
`<header>`	Logo, nome do sistema e botão de tema.
`<ol id="etapas">`	A barra "1. Vaga e período, 2. Seus dados, 3. Pagamento". O JS marca qual está ativa.
`form#tela-vaga`	Etapa 1: campos de período, a planta (div onde o JS desenha as vagas) e o resumo com o botão "Continuar".
`section#tela-confirmar`	Etapas 2 e 3: resumo à esquerda e, à direita, `form#form-dados` ou `form#form-pagamento` (só um aparece).
`section#minhas`	Lista de reservas, contador e botão "Limpar reservas".
`dialog#modal-saida`	Janela (modal) para simular a saída. Usa a tag nativa `<dialog>`.

Recursos do HTML que usamos

- `type="date"`, `type="time"`, `type="datetime-local"`: o navegador já mostra o seletor de data e hora.
- `type="file"` com `accept`: escolher o anexo da credencial.
- `hidden`: esconde elementos (por exemplo, o campo de credencial só aparece em vaga acessível).
- `<dialog>`: janela modal sem precisar de biblioteca.

5. O arquivo style.css

Temas por variáveis

As cores ficam em variáveis no topo (`--destaque`, `--superficie`, `--texto` etc.). O tema escuro apenas **redefine essas variáveis**, e o resto do CSS não muda.

```
:root { --fundo: #f5f6f7; --destaque: #c44100; ... }
:root[data-tema="escuro"] { --fundo: #101214; --destaque: #ff7b29; ... }
```

O JavaScript troca o tema apenas mudando o atributo `data-tema` do elemento raiz.

A planta do estacionamento

A planta é uma caixa com `position: relative`. Dentro dela, três fileiras com `position: absolute` e medidas em porcentagem, o que mantém o desenho proporcional. Cada fileira usa **CSS Grid**:

- Fileira de cima: 7 colunas (pilha de 3 motos + V1 a V6).
- Fileira do meio: 8 colunas por 2 linhas (D1, V7 a V13, D2, V14 a V20).
- Fileira de baixo: 7 colunas (pilha de 3 motos + V21 a V26).

Estados da vaga

Visual	Significado	Regra CSS
Branca	Disponível	`.vaga`
Laranja	Selecionada	`.vaga.selecionada`
Cinza	Reservada ou período inválido	`.vaga:disabled`

Layout responsivo

No computador, a planta fica à esquerda e período/resumo à direita (`grid-template-areas`). Abaixo de 1000 px de largura, tudo vira uma coluna: período, planta e resumo. A planta ganha rolagem horizontal para caber no celular.

6. O arquivo script.js

É dividido em blocos, cada um com um comentário no código. Esta é a ordem para explicar.

6.1 Dados e constantes

```
const TARIFAS = { hora: 8, diaria: 35, horaExtra: 5 };
const MS_POR_HORA = 36e5;   // 1 hora em milissegundos
let reservas = [];   // todas as reservas (vem do localStorage)
let vagaSelecionada = null; // vaga clicada na planta
```

Para mudar os preços, basta alterar `TARIFAS`. As datas são trabalhadas em **milissegundos**, o que torna fácil somar e comparar horários.

6.2 Desenhando a planta

As 34 vagas não estão escritas no HTML. A função `criarVaga()` cria um botão para cada uma, e os laços `carros()` e `motos()` repetem o processo. Os botões ficam guardados na lista `vagas`, e assim o JS consegue pintar ou bloquear cada um depois.

6.3 Período e valor

```
function periodo() {
  const inicio = +new Date(`${dia}T${entrada}`);
  if (modalidade() == "diaria") return { inicio, fim: inicio + 24*MS_POR_HORA, valor: 35 };
  const fim = +new Date(`${dia}T${saida}`);
  if (!(fim > inicio)) return { erro: "A saída precisa ser depois da entrada..." };
  return { inicio, fim, valor: Math.ceil((fim-inicio)/MS_POR_HORA) * 8 };
}
```

Devolve o início, o fim e o valor, ou uma mensagem de `erro`. `Math.ceil` arredonda as horas para cima: 2h30 cobra 3 horas, ou seja, R$ 24,00.

6.4 A regra mais importante: conflito de horário

```
function conflito(periodoAtual, campo, valor) {
  if (periodoAtual.erro) return false;
  return reservas.some(reserva => reserva.status == "ativa" && reserva[campo] == valor
    && periodoAtual.inicio < reserva.fim && periodoAtual.fim > reserva.inicio);
}
```

Dois períodos se sobrepõem quando **cada um começa antes de o outro terminar**. A mesma função serve para vaga (`campo = "vaga"`) e para placa (`campo = "placa"`). Reservas coladas (uma termina às 10h e outra começa às 10h) não conflitam.

> **Exemplo:** reserva existente 08h às 10h. Nova das 09h às 11h: começa (09h) antes do fim da antiga (10h) e termina (11h) depois do começo dela (08h), então conflita. Nova das 10h às 12h: 10h não é menor que 10h, então não conflita.

6.5 Etapa 1: atualizar()

Roda a cada mudança no formulário. Ela recalcula o período, percorre as 34 vagas bloqueando as ocupadas, marca a selecionada, atualiza o resumo (entrada, saída, duração, total) e habilita ou desabilita o botão "Continuar". Se o novo período ocupar a vaga já escolhida, a escolha é desfeita com um aviso.

6.6 Navegação entre as etapas: mostrar()

Uma única função mostra e esconde as telas conforme o número da etapa (0, 1 ou 2) e atualiza a barra de progresso. O botão "Voltar" apenas chama `mostrar(etapa - 1)`.

6.7 Etapa 2: validação dos dados

Campo	Regra
Nome	Pelo menos nome e sobrenome, só letras, espaços e pontuação de nomes.
Telefone	DDD com 10 dígitos (fixo) ou 11 (celular). Aceita o +55 e formata como (11) 99999-8888.
Placa	Padrão antigo (ABC1234) ou Mercosul (ABC1D23), em maiúsculas.
Placa em uso	Usa `conflito()` para impedir duas reservas da mesma placa no mesmo horário.
Credencial	Só nas vagas D1 e D2: precisa ter tipo permitido, conteúdo e no máximo 5 MB.

As validações usam **expressões regulares**. Por exemplo, a placa: `/^[A-Z]{3}[0-9][A-Z0-9][0-9]{2}$/` significa 3 letras, 1 número, 1 letra ou número e 2 números. Se algo estiver errado, a função `erroDados()` mostra a mensagem e põe o cursor no campo com problema.

6.8 Etapa 3: pagamento e criação da reserva

O objeto `PAGAMENTOS` descreve as 4 formas (nome, texto e se é "no local"). Ao escolher uma, a tela mostra a descrição, o QR Code de demonstração no caso do Pix e o texto do botão. Ao confirmar, o sistema **confere a disponibilidade de novo** (outra aba poderia ter reservado a vaga) e só então cria o objeto da reserva:

```
{ id, vaga, tipo, nome, telefone, placa, credencialAnexada,
  pagamento, pagamentoSimulado, modalidade, inicio, fim,
  valor, adicional: 0, status: "ativa" }
```

Esse objeto entra no array `reservas` e é salvo com `salvar()`.

6.9 Salvamento: localStorage

```
localStorage.setItem(CHAVE_ARMAZENAMENTO, JSON.stringify(reservas));   // salvar
reservas = JSON.parse(localStorage.getItem(CHAVE_ARMAZENAMENTO) || "[]"); // carregar
```

O `localStorage` guarda texto no navegador. Por isso o array é convertido em texto (`JSON.stringify`) ao salvar e de volta em objetos (`JSON.parse`) ao carregar. Usamos `try/catch` porque alguns navegadores bloqueiam o armazenamento.

6.10 Minhas reservas, cancelar e saída

- **renderizar()** monta a lista. Cada reserva vira um cartão com os botões "Simular saída" e "Cancelar reserva" (somente se estiver ativa).
- Os cliques usam **delegação de eventos**: um único `onclick` na lista descobre qual botão foi clicado pelo atributo `data-cancelar` ou `data-saida`.
- **Cancelar** pede confirmação, muda o status para "cancelada" e a vaga fica livre.
- **Saída**: calcula os minutos depois do fim contratado, arredonda para horas iniciadas e multiplica por R$ 5,00.

```
const minutosExcedidos = Math.max(0, Math.ceil((saidaReal - reservaEmSaida.fim) / 60000));
const adicional = Math.ceil(minutosExcedidos / 60) * TARIFAS.horaExtra;
```

Exemplo: contratado até 10h, saiu às 11h20. São 80 minutos, que viram 2 horas iniciadas, ou seja, R$ 10,00 de adicional.

6.11 Segurança básica

O nome digitado passa pela função `escaparHtml()` antes de entrar no HTML. Sem isso, alguém poderia digitar código no campo nome e ele seria executado na página (ataque chamado **XSS**).

7. Limites do protótipo

- Sem servidor e sem banco: os dados existem só no navegador de quem usou e não são compartilhados entre pessoas.
- O pagamento é simulado. Não existe integração com banco, Pix ou Google.
- A credencial é só conferida no momento do envio (tipo e tamanho) e depois descartada. A autenticidade do documento não é verificada.
- Se duas pessoas usarem o mesmo computador, as reservas aparecem para ambas.

Como evoluiria: um servidor (por exemplo Node.js) com banco de dados para as reservas compartilhadas, login de usuários, integração com um gateway de pagamento real e armazenamento seguro das credenciais.

8. Perguntas que o professor pode fazer

Pergunta	Resposta curta
Por que não usaram framework ou banco de dados?	O objetivo foi aprender a base da web (HTML, CSS e JS) e manter o projeto simples de abrir e entender. O `localStorage` atende um protótipo.
Como evitam duas reservas na mesma vaga e horário?	A função `conflito()` compara os períodos. A vaga é bloqueada na planta e a verificação é repetida ao confirmar.
Onde os dados ficam salvos?	No `localStorage` do navegador, como texto JSON.
Como calculam o valor?	Horas arredondadas para cima vezes R$ 8, ou R$ 35 fixos na diária. Atraso: R$ 5 por hora iniciada.
O que é uma expressão regular?	Um padrão que descreve a forma de um texto. Usamos para validar placa e telefone.
O que acontece se limpar o navegador?	As reservas são perdidas, pois estão só ali. Num sistema real ficariam no servidor.
Como o tema escuro funciona?	O JS muda o atributo `data-tema`. O CSS redefine as variáveis de cor.
O pagamento é real?	Não. Cartão e dinheiro ficam pendentes; Google Wallet e Pix são só simulação.
Por que a versão "simples"?	A versão original tem cerca de 3.500 linhas por causa de acessibilidade avançada e validações extras. Mantivemos as mesmas funções em cerca de 850 linhas (com muitos comentários), mais fáceis de explicar.

9. Sugestão de divisão da apresentação

Pessoa	Parte	O que mostrar
1	Visão geral e `index.html`	Objetivo, tecnologias, estrutura da página e das 3 etapas.
2	`style.css`	Variáveis, tema escuro, desenho da planta com Grid, layout responsivo.
3	Planta, período e conflito	`criarVaga`, `periodo`, `conflito` e `atualizar`.
4	Etapas 2 e 3	Validações com regex, pagamento, criação da reserva e `localStorage`.
5	Reservas, saída e demonstração	`renderizar`, cancelar, taxa de atraso, limites e demonstração ao vivo.

Roteiro da demonstração ao vivo

1. Abrir o site: mostrar a planta com todas as vagas livres e trocar o tema.
2. Escolher a vaga V1 e fazer a reserva completa com Pix.
3. Voltar à etapa 1 com o **mesmo horário**: mostrar a V1 bloqueada.
4. Tentar reservar outra vaga com a **mesma placa**: mostrar o erro.
5. Escolher D1 para mostrar o pedido de credencial.
6. Em "Minhas reservas", simular uma saída com atraso e mostrar a taxa extra.
7. Cancelar uma reserva e mostrar a vaga livre de novo. Recarregar a página para provar que os dados ficaram salvos.